Room Booking Platform - Design

1. Motivation

The goal of this task is to design and implement a scalable and fault-tolerant Room Booking Platform.

The platform should allow users to register, login, search for rooms, and book a room.
From a design perspective, it should also support booking cancellation and booking history retrieval, even though those two flows are not required in the implementation.

The main challenge in this system is not just exposing APIs. The real challenge is making sure the booking flow stays correct under concurrent requests, while still keeping the system scalable and simple enough for this scope.

2. Scope

2.1 In Scope

The implemented solution includes:

- User registration
- User authentication
- Room search
- Room booking
- API validation and error handling
- Rate limiting
- Data consistency for the booking flow
- Concurrency handling to prevent double booking

2.2 Design Scope

The following are included in the system design, but are not fully implemented:

- Multi-region strategy
- Load balancer placement and traffic routing
- Booking cancellation
- Booking history retrieval
- Fault tolerance considerations beyond the local implementation scope

2.3 Out of Scope

The following are not implemented:

- Booking cancellation flow
- Booking history retrieval flow
- Real notification delivery such as booking confirmation emails
- Actual multi-region deployment

3. Assumptions and Constraints

- The system is implemented as two services only: a frontend service and a backend service.
- The implemented flows include user registration, room search, and room booking.
- Booking cancellation and booking history retrieval are not implemented, but they are supported in the design.
- Multi-region deployment and load balancer strategy are included at the design level only.
- The system should support scalability, rate limiting, and safe concurrent booking behavior.
- Only authenticated users can create bookings.
- Time values are handled in a consistent timezone format (such as UTC).
- PostgreSQL is the system of record for booking decisions.
- Redis is used as a supporting component for caching, rate limiting, and idempotency, but not as the source of truth.

4. High-Level Architecture

The Room Booking Platform is designed as a two-service system:

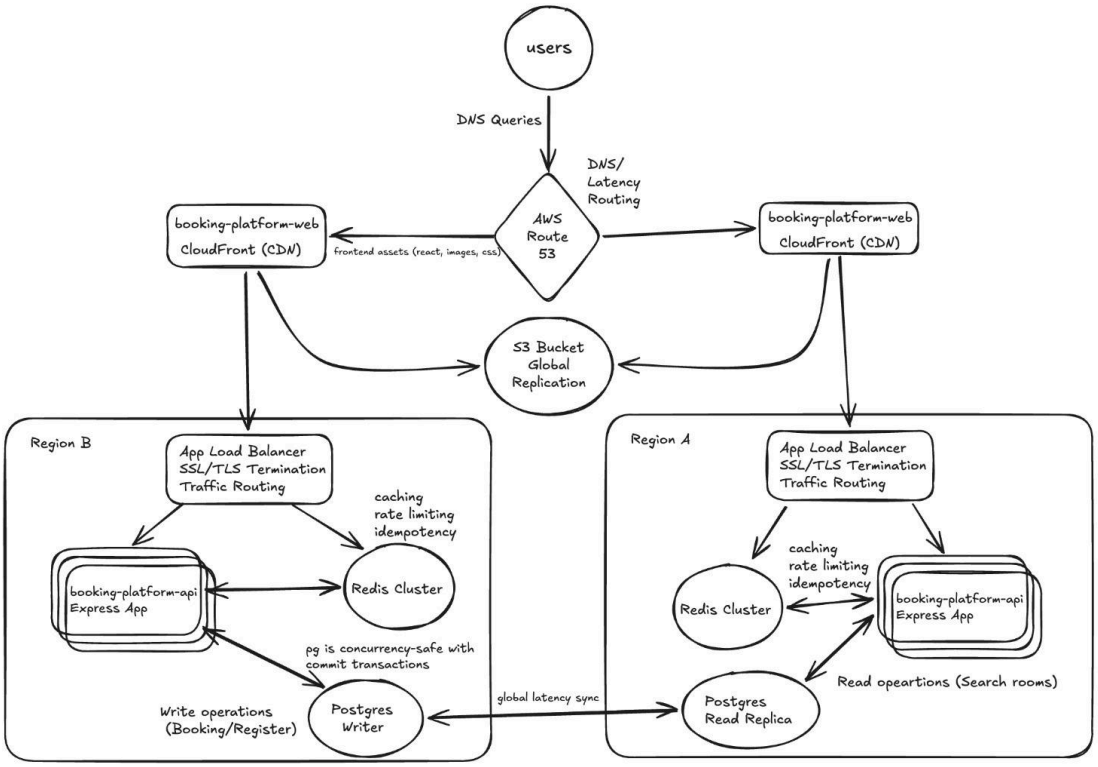
1. **Frontend Service**
2. **Backend Service**

4.1 Architecture Goals

The architecture is designed with the following goals in mind:

- support registration, search, and booking
- prevent double booking under concurrent requests
- keep the backend stateless and horizontally scalable
- scale search traffic efficiently
- support future multi-region deployment and load balancing

4.2 High-Level Architecture Diagram



4.3 Main Components

Frontend Service

The frontend service is a React + TypeScript web application responsible for the user facing flows.

Its responsibilities include:

- user registration
- user login
- room search form
- displaying room search results
- initiating booking requests
- showing loading, success, and error states

The frontend does not enforce booking consistency rules. Its job is to collect user input, call the backend APIs, and display the result.

Backend Service

The backend service is a Node.js + TypeScript API that contains the system's business logic.

Its responsibilities include:

- registering and authenticating users
- validating API requests
- exposing room search APIs
- handling booking creation
- persist data with PostgreSQL
- using Redis for caching, idempotency, and rate limiting

The backend is designed as a **stateless service**, which means any instance can handle any incoming request. This makes it easy to scale horizontally behind a load balancer.

PostgreSQL

PostgreSQL is the primary database and the source of truth of the system.

It stores:

- users
- rooms
- rooms_amenities
- bookings

PostgreSQL is used because booking systems require strong consistency and safe concurrent writes. Since preventing overlapping bookings is the most critical rule in the system, a relational database with transactions is the safest and clearest choice.

Redis

Redis is used as a supporting infrastructure component.

It is used for:

- API rate limiting
- caching repeated search queries
- caching room related data when useful
- storing idempotency keys for retry safe booking creation

Redis improves performance and protects the system under load, but it is not the source of truth for booking decisions.

Load Balancer

A load balancer is part of the design scope.

Its responsibilities include:

- distributing traffic across healthy backend instances
- improving availability
- supporting future scaling of stateless backend nodes

It is not implemented in the assignment, but the system is designed so it can run behind one without changes to the core logic.

Multi-Region Readiness

Multi-region deployment is also part of the design scope only.

At the design level:

- traffic can be routed to the nearest region
- read heavy traffic such as search can benefit from regional locality
- write sensitive booking flows should be routed carefully to preserve consistency

This is not implemented in the assignment, but the architecture supports it conceptually.

4.4 Fault Tolerance Considerations

The system is also designed with fault tolerance in mind.

Examples:

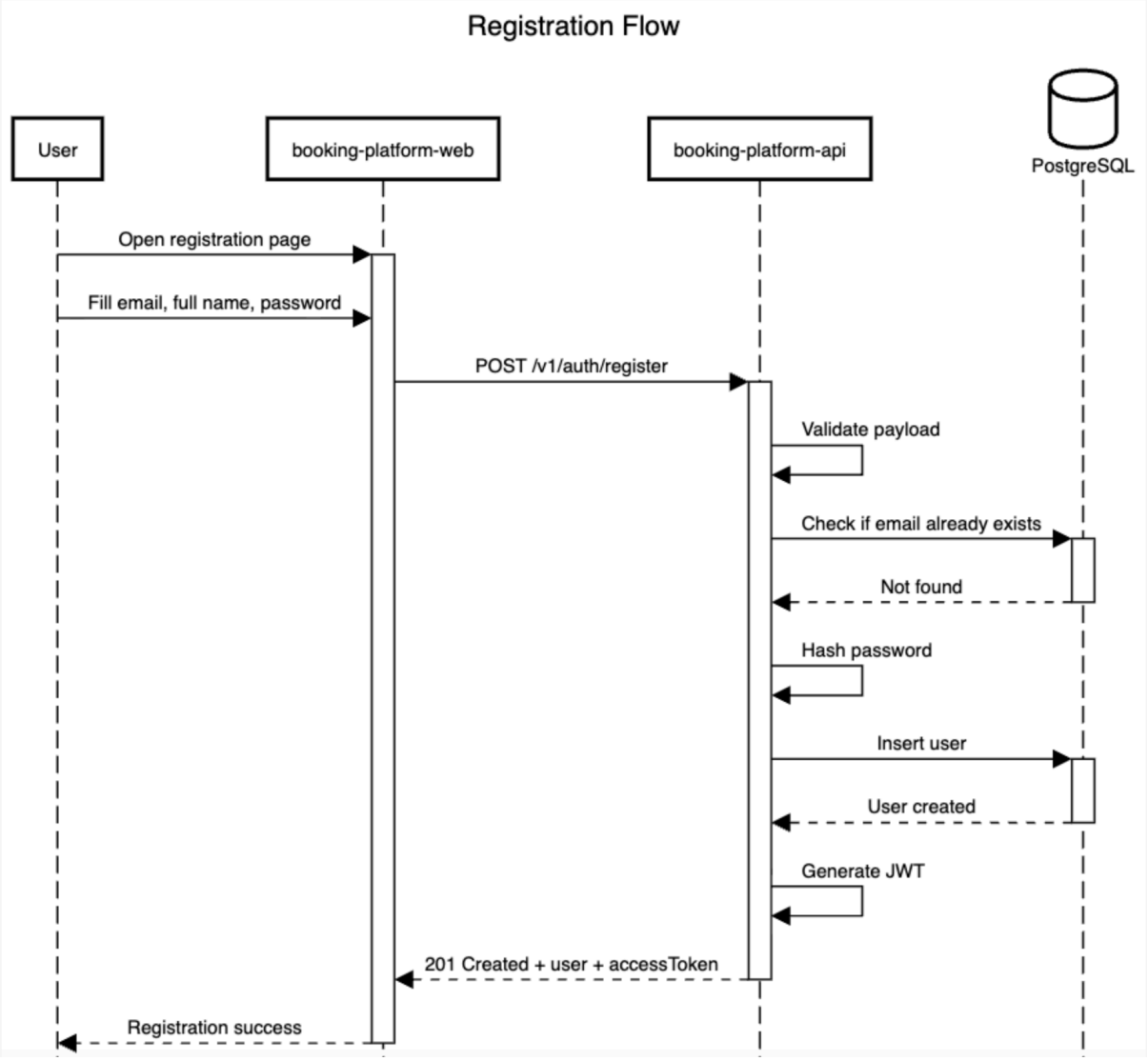
- if one backend instance fails, other instances can still serve traffic
- if Redis is unavailable, the system can still work using PostgreSQL directly, though with reduced performance
- if a booking request is retried because of a timeout, idempotency protection helps avoid duplicate bookings
- multi-region failover is not implemented, but the design leaves room for it in the future

This means the system is designed to fail gracefully where possible, instead of depending on every component always being available.

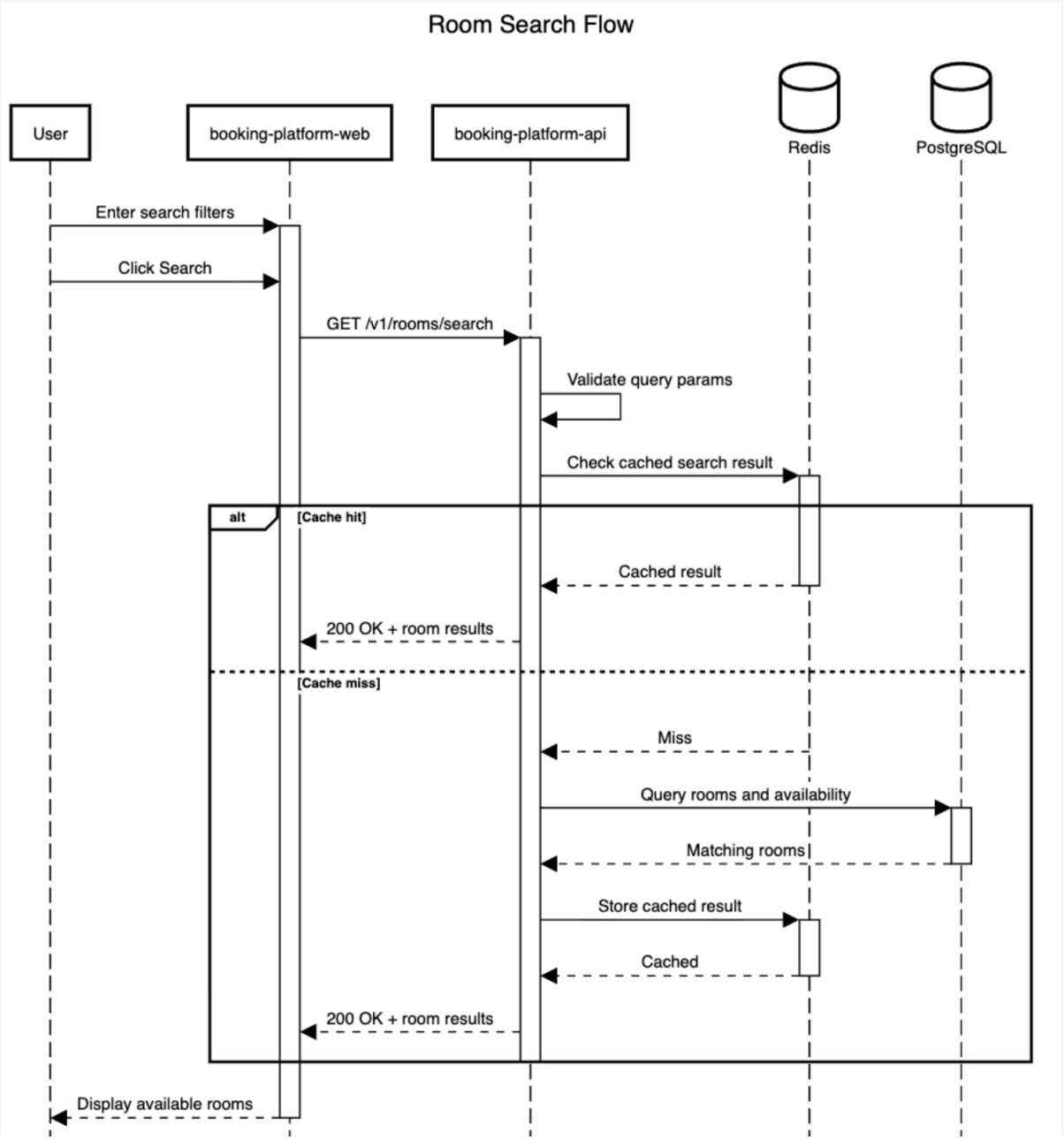
5. Request Flows

The main flows in the implemented system are registration, room search, and booking.

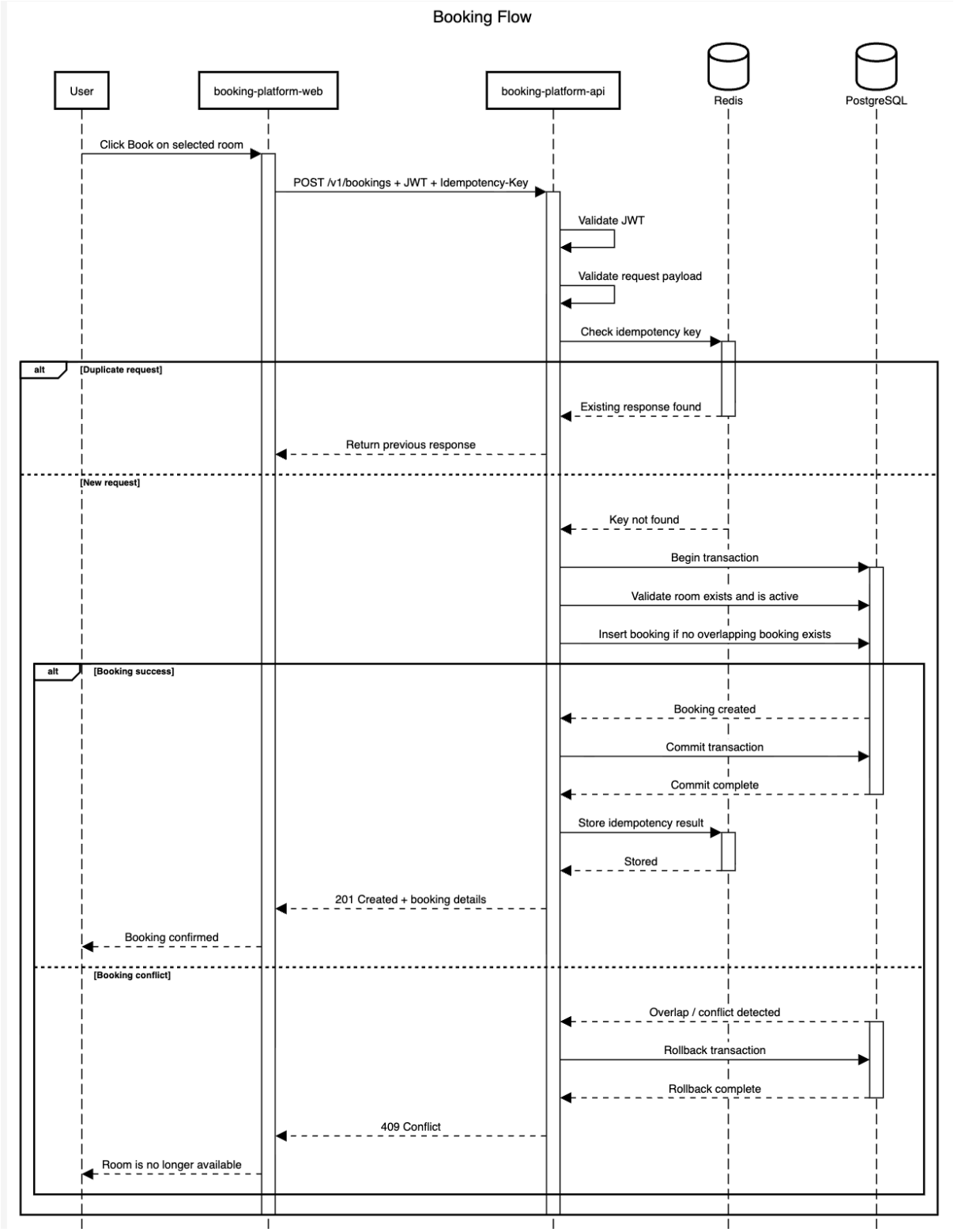
5.1 Registration Sequence Diagram



5.2 Room Search Sequence Diagram



5.3 Booking Sequence Diagram



6. API Design

The backend exposes a REST API over HTTPS. REST is a good fit here because the required operations map naturally to clear resources and endpoints.

Authentication is based on JWT bearer tokens.

6.1 Authentication

POST `/v1/auth/register`

Creates a new user account.

Request

```
{
 "email": "user@example.com",
 "password": "StrongPass123!",
 "fullName": "AAA bbbb"
}
```

Success response

```
{
 "user": {
   "id": 1,
   "email": "user@example.com",
   "fullName": "AAA bbbb"
 },
 "accessToken": "jwt-token"
}
```

Possible errors:

- `400 Bad Request` - invalid input
- `409 Conflict` - email already exists

POST `/v1/auth/login`

Authenticates a user and returns an access token.

Request

```
{
 "email": "user@example.com",
 "password": "StrongPass123!"
}
```

Success response

```
{
 "accessToken": "jwt-token"
}
```

Possible errors:

- `400 Bad Request`

- `401 Unauthorized`

6.2 Rooms

GET `/v1/rooms/search`

Searches for rooms using filters.

Query parameters

- `location`
- `capacity`
- `startTime`
- `endTime`
- `amenities`
- `page`
- `limit`

Example

GET /v1/rooms/search?location=TLV&capacity=4&startTime=2026-03-12T09:00:00Z&endTime=2026-03-12T10:00:00Z&page=1&limit=20

Success response


```
{
 "items": [
   {
     "id": 101,
     "name": "Std Room 1",
     "location": "TLV",
     "capacity": 6,
     "amenities": ["TV", "WHITEBOARD"],
     "isAvailable": true
   }
 ],
 "page": 1,
 "limit": 20,
 "total": 1
}
```


Possible errors:

- `400 Bad Request`

GET `/v1/rooms/:roomId`

Returns room details.

Success response

```
{
 "id": 101,
 "name": "Std Room 1",
 "location": "TLV",
 "capacity": 6,
 "amenities": ["TV", "WHITEBOARD"],
 "status": "ACTIVE"
}
```

Possible errors:

- `404 Not Found`

6.3 Bookings

POST `/v1/bookings`

Creates a booking for an authenticated user.

Headers

Authorization: Bearer <access_token>
Idempotency-Key: <uuid>

Request

```
{
 "roomId": 101,
 "startTime": "2026-03-12T09:00:00Z",
 "endTime": "2026-03-12T10:00:00Z"
}
```

Success response

```
{
 "id": 1001,
 "userId": 1,
 "roomId": 101,
 "startTime": "2026-03-12T09:00:00Z",
 "endTime": "2026-03-12T10:00:00Z",
 "status": "CONFIRMED"
}
```

Possible errors:

- `400 Bad Request`
- `401 Unauthorized`
- `404 Not Found`
- `409 Conflict`

Example conflict response:

```
{
 "error": {
   "code": "ROOM_ALREADY_BOOKED",
   "message": "The room is no longer available for the selected time range."
 }
}
```

6.4 Validation Rules

The API validates all incoming requests before running business logic.

Examples:

- email must be valid
- password must meet minimum rules
- `startTime` must be earlier than `endTime`
- room must exist and be active
- pagination values must be positive integers

6.5 Rate Limiting Strategy

Rate limiting is enforced at the backend layer using Redis.

- `POST /v1/auth/register` → 5 requests/minute/IP
- `POST /v1/auth/login` → 5 requests/minute/IP
- `GET /v1/rooms/search` → 60 requests/minute/user or IP
- `POST /v1/bookings` → 10 requests/minute/user

If a limit is exceeded, the backend returns:

- `429 Too Many Requests`

This protects the system from brute force attempts, accidental flooding, and abusive traffic.

7. Database Schema

The platform uses PostgreSQL as its primary database and source of truth.
The core entities are:

- users

- rooms
- room_amenities
- bookings

A relational database is preferred because this system needs strong consistency, especially in the booking flow.

7.1 Schema Design Principles

The schema is designed according to the following principles:

- PostgreSQL is the final source of truth for booking decisions
- hot paths such as booking conflict checks should be optimized
- duplicated sources of truth should be avoided where possible

7.2 Entity Relationships

- one user can create many bookings
- one room can have many bookings over time
- one booking belongs to exactly one user and one room

Logical relationships:

- `users 1:N bookings`
- `rooms 1:N bookings`

7.3 Users Table

The `users` table stores registered users.

```
CREATE TABLE users (
 id BIGSERIAL PRIMARY KEY,
 email VARCHAR(255) NOT NULL UNIQUE,
 password_hash VARCHAR(255) NOT NULL,
 full_name VARCHAR(255) NOT NULL,
 created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
 updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```

Indexing

- unique index on `email`

This supports registration and login by email efficiently.

7.4 Rooms Table

The `rooms` table stores the room inventory.

```
CREATE TABLE rooms (
 id BIGSERIAL PRIMARY KEY,
 name VARCHAR(255) NOT NULL,
 location VARCHAR(255) NOT NULL,
 capacity INT NOT NULL,
 status VARCHAR(32) NOT NULL,
 region_code VARCHAR(32) NOT NULL,
 created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
 updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```

Field rationale

- `name` identifies the room
- `location` supports search
- `capacity` supports filtering
- `status` allows values such as `ACTIVE`, `INACTIVE`, `MAINTENANCE`
- `region_code` supports future multi-region ownership

Indexing

- index on `(location, status)`
- optional compound index on `(location, status, capacity)`

7.5 Room Amenities

```
CREATE TABLE room_amenities (
 room_id BIGINT NOT NULL REFERENCES rooms(id),
 amenity VARCHAR(64) NOT NULL,
 PRIMARY KEY (room_id, amenity)
);
```

Indexing

- primary key on `(room_id, amenity)`
- optional index on `(amenity)`

This supports both:

- reading amenities per room
- filtering rooms by amenity

7.6 Bookings Table

The `bookings` table stores the reservations and is the most important table in the system.

```
CREATE TABLE bookings (
```

```
 id BIGSERIAL PRIMARY KEY,
 user_id BIGINT NOT NULL REFERENCES users(id),
 room_id BIGINT NOT NULL REFERENCES rooms(id),
 start_time TIMESTAMPTZ NOT NULL,
 end_time TIMESTAMPTZ NOT NULL,
 status VARCHAR(32) NOT NULL,
 created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
 updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
 CHECK (start_time < end_time)
);
```

Field rationale

- `user_id` links the booking to the user
- `room_id` links the booking to the room
- `start_time` and `end_time` define the booking window
- `status` supports future flows such as cancellation
- the check constraint prevents invalid time ranges

Indexing

```
CREATE INDEX idx_bookings_room_start_time
ON bookings (room_id, start_time);
```

Optional future index:

```
CREATE INDEX idx_bookings_user_created_at
ON bookings (user_id, created_at DESC);
```

This supports:

- availability checks by room and time range
- future booking history queries

7.7 Availability Model

Availability is not stored as a separate writable table in the initial design.

Instead, it is derived from:

- the selected room
- the requested time range
- existing active bookings

A room is considered available if there is no overlapping active booking for the requested time range.

This design avoids keeping two writable sources of truth in sync.
If needed later, availability can be placed into a cache or read optimized view for faster search performance.

7.8 Data Consistency Considerations

The most important consistency rule is:

> the same room must not have overlapping active bookings for the same time range

Other important rules include:

- a booking must reference an existing user
- a booking must reference an existing room
- `start_time` must be earlier than `end_time`
- inactive rooms must not be bookable

8. Concurrency Handling

Preventing double booking is the most critical consistency requirement in the system.

A simple application level flow such as “check availability first, then insert booking” is not safe enough. Two concurrent requests may both read the room as available before either one writes to the database, which can lead to a race condition and conflicting bookings.

Because the backend is stateless and can run on multiple instances behind a load balancer, this problem cannot be solved safely with in-memory locking inside one API instance. The consistency guarantee must be enforced at the database and transaction level.

The booking flow therefore relies on:

- request validation
- short database transactions
- database level overlap protection
- idempotency support for retry safety

When a booking request arrives, the backend validates the authenticated user, request body, room existence, and room status. It then starts a database transaction and attempts to create the booking only if no overlapping active booking already exists for the same room and requested time range.

Two bookings conflict if their time ranges overlap. A common overlap condition is:

NOT (existing.end_time <= requested_start OR existing.start_time >= requested_end)

In other words, if an existing booking does not end before the requested booking starts, and does not start after the requested booking ends, then the time ranges overlap and both bookings cannot be confirmed.

The main design principle is that the database must be the final authority that prevents conflicting writes. This can be implemented through transactional guarded inserts or database level constraints. The exact implementation can vary, but the important point is that concurrency correctness is enforced in the persistence layer, not only in application code.

This means that if two requests arrive at almost the same time, only one of them can succeed. The other one will fail with a `409 Conflict` response.

In addition to concurrent requests from different users, duplicate booking attempts may also happen because of retries, timeouts, or repeated clicks. To protect against this, the booking endpoint accepts an `Idempotency-Key` header. If the same logical request is retried with the same key, the backend returns the original result instead of creating a duplicate booking. These keys can be stored in Redis with a short TTL together with the original response.

From a performance point of view, the concurrency strategy should avoid broad locking and keep transactions short. Conflict checks should be scoped to the relevant `room_id` and time range, supported by proper indexes on the `bookings` table. This keeps contention local and allows the system to remain safe under load without unnecessary locking.

9. Scalability Strategies

The system is designed to scale differently for its two main traffic patterns: **room search** and **booking creation**.

This distinction is important because search is expected to be read heavy, while booking is write sensitive and must prioritize correctness over raw throughput.

9.1 Search Scalability

Room search is expected to be the highest volume operation in the platform.

To scale search efficiently, the system uses:

- Redis caching for repeated queries
- database indexing for common filters
- pagination to limit result size

This reduces load on PostgreSQL and improves response time.

In a larger future version, search could also use:

- read replicas
- a dedicated search engine

For this scope, PostgreSQL with proper indexes and Redis caching is sufficient.

9.2 Booking Scalability

Booking is different from search because correctness matters more than speed alone.

The backend service can scale horizontally because it is stateless. Multiple instances can run behind a load balancer and process traffic in parallel.

At the same time, booking consistency remains protected by PostgreSQL through short transactions and overlap validation.

To reduce unnecessary load, the system also uses:

- rate limiting
- idempotency keys

This keeps the booking flow both safe and stable under load.

9.3 Horizontal Scaling of the Backend

The backend can be scaled by running multiple stateless instances behind a load balancer.

which include:

- higher throughput
- better availability
- easier rolling deployments
- simpler recovery from instance failures

Because request specific state is not stored in process memory, scaling out does not break consistency.

9.4 Database Scaling Considerations

For our scope, a single primary PostgreSQL database is enough.

As the system grows, database scaling can evolve in phases:

1. single primary database
2. read replicas for search heavy traffic

9.5 Redis as a Scalability Enabler

Redis improves scalability without becoming a source of truth.

It is used for:

- rate limiting
- short lived search cache
- idempotency key storage

If Redis is unavailable, the system can still work using PostgreSQL directly, although performance may be reduced.

9.6 Load Balancer and Multi-Region Readiness

At the design level, the system is ready to run behind a load balancer and be extended to multiple regions in the future.

The load balancer improves capacity and resilience by distributing traffic across healthy backend instances.

In a future multi-region version:

- search can benefit from regional locality
- booking writes should be routed carefully to preserve consistency

This is an important separation: search can scale broadly, but booking must stay strongly consistent.

10. Optional Components

Although not required in the implementation, several supporting components would be important in a production version of the platform.

Monitoring

Useful metrics include:

- request rate
- error rate
- booking success rate
- booking conflict rate
- cache hit ratio
- database query latency
- rate limit rejections

These metrics help understand performance and detect issues early.

Logging

The backend should produce structured logs.

Useful fields include:

- correlation ID
- user ID
- room ID
- booking ID
- endpoint
- response status code
- error code

This makes debugging and operational analysis much easier.

Notifications

After a successful booking, the system could send a confirmation email.

This should be handled asynchronously so it does not slow down the booking transaction itself.

A typical flow would be:

1. booking succeeds
2. backend publishes an event or creates a background job
3. notification worker sends the email

Health Checks

The backend should expose health endpoints so infrastructure can check whether instances are healthy and ready to serve traffic.

Examples:

- liveness check
- readiness check
- database connectivity check
- optional Redis connectivity check